

Accessing MongoDB from Python

Sure! Let's talk about the MongoClient and how it helps us interact with MongoDB in Python.

The MongoClient is like a friendly messenger that connects your Python program to a MongoDB database. Imagine you want to send a letter (data) to a friend (the database). The MongoClient is the postal service that ensures your letter reaches your friend safely. To use it, you first import it from a library called pymongo, which is like a toolkit specifically designed for working with MongoDB in Python. Once you create a MongoClient, you can access your database and collections, just like opening a mailbox to find your letters.

Now, once you have access to your database, you can perform various operations, like adding new students to a class (inserting documents), checking who is in the class (reading documents), updating their information (updating documents), or even removing someone from the class (deleting documents). For example, if you want to add a new student, you would use the `insert_one` function, which is like writing a new letter and sending it off. If you want to update a student's email, you can use the `update_one` function, which allows you to change just a part of the letter without rewriting the whole thing.

MongoClient

A class that helps you interact with MongoDB

```
from pymongo import MongoClient
uri = "mongodb://USER:PASSWORD@uri/test"
client = MongoClient(uri)
campusDB = client.campusManagementDB
students = campusDB.students
```

- The MongoClient is a class that helps you interact with MongoDB. First, we import MongoClient from pymongo, which is an official MongoDB driver for Python. The 'uri' tells us the address where MongoDB is. Then we create MongoClient, which we should only do once in our code. The next part gets the object pointing to the campus management database, And finally, from the campus management database object, we point to the 'students' collection.

Create

```
students.insert_one({ firstName: "John",
                      "lastName": "Doe",
                      "email": "john.doe@email.com",
                      "studentId": 20217484
                    })
```

- Now that we have a reference to the 'students' collection, Let's create a student document. For this, we call the 'insert_one' function, which takes the document we are trying to insert as its input.

Create Many

```
students_list = [  
    {"firstName": "Sarah", "lastName": "Jane",  
     "email": "sarah@mail.com", "studentId": 20215124},  
    {"firstName": "Peter", "lastName": "Parker",  
     "email": "peter@web.com", "studentId": 20218873}]
```

```
students.insert_many(students_list)
```

- We can also do a bulk insert by calling the 'insert_many' function. This function takes a list instead as its input. Now let's do some Read operations.

Read

```
students.find_one()
```

```
students.find_one({"email": "sarah@mail.com"})
```

```
students.find({"lastName": "Doe"})
```

```
students.count_documents({"lastName": "Doe"})
```

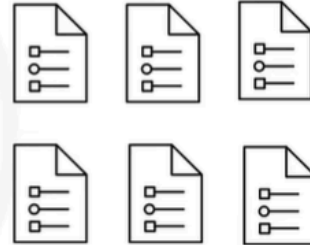
- In the first one, we call 'find_one', without any filter criteria. This will return the first document in natural order, which is the order in which the database refers to documents on disk. Then we find the first student with an email. If more than one document matches the criteria, only the first one will be read, because we are using the 'find_one' function.
- To get all the documents, we do a 'find'. Now we are retrieving all students with the lastname of Doe. And finally, if we just want to count how many students there are with the lastname Doe, we call the 'count' function. This count will happen on MongoDB, and only the number value will be returned to the client.

Print read documents

```
from bson.json_util import dumps
```

```
cursor = students.find({"lastName": "Doe"})
```

```
print(dumps(cursor, indent=4))
```



Print read documents

```
from bson.json_util import dumps

cursor = students.find({"lastName": "Doe"})
print(dumps(cursor, indent=4))
```



Print read documents

```
from bson.json_util import dumps

cursor = students.find({"lastName": "Doe"})
print(dumps(cursor, indent=4))
```

- When we do a 'find' on a collection in MongoDB, we get a cursor back. This cursor points to our documents in MongoDB. If you have run this using mongo shell, the shell retrieves those documents for you behind the scenes. To retrieve documents from our cursor, we call python 'dumps', which will exhaust our cursor to retrieve all the documents.

Replace

```
student = students.find_one({"lastName": "Doe"})
student["onlineOnly"] = True
student["email"] = "johnd@campus.edu"

students.replace_one({"lastName": "Doe"}, student)
```

- Let's retrieve a student and make some changes to the document. We are creating a new field to identify whether students are on campus or online only.
- We will also update the email with the campus email address. Once all changes are made, we will call the 'replace' function, Which takes its first argument as the filter criteria to find a document with last name of Doe, And the second argument is the updated python object.

Update

```
changes = {"$set": {"onlineOnly": True,
"email": "johnd@campus.edu"
}}

students.update_one({"lastName": "Doe"}, changes)

students.update_many({}, {"$set": {"onlineOnly": True}})
```

- In the Replace example, we were sending the whole document back with the new changes. For larger documents, this will take a lot of transfer time between the client and the database. We can use MongoDB's in-place updates for small changes.
- We do this by constructing a change document, which in our case only has two assignments, which come under '\$set'. And with this in-place update, we don't even need to retrieve the student document. To update all of the students to online only due to lockdown, we can run the 'update_many' function without any filter criteria, and change all documents

Delete

```
students.delete_one({"studentId": 20218873})
```

```
students.delete_many({"graduatedYear": 2019})
```

- Just like update and find, the 'delete_one' and 'delete_many' functions take filter criteria as the first argument, And these functions will delete one or many documents in the given collection matching the criteria.

Recap

In this video, you learned that:

- MongoClient is a class that helps you interact with MongoDB
- MongoClient is imported from pymongo, the official MongoDB driver for Python
- You can perform single or bulk inserts
- You can replace whole documents
- You can perform in-place updates
- You can delete one or many documents